

GRAPH PROBLEMS - JAVA (NOTEPAD STYLE)

Problem 1: Social Network

```
class Graph{
    int v;
    ArrayList<ArrayList<Integer>> adj;

    Graph(int v){
        this.v=v;
        adj=new ArrayList<>();
        for(int i=0;i<v;i++)
            adj.add(new ArrayList<>());
    }

    void addEdge(int u,int v){
        adj.get(u).add(v);
        adj.get(v).add(u);
    }

    void friends(int u){
        for(int x:adj.get(u))
            System.out.print(x+" ");
    }

    boolean isConnected(int u,int v){
        return adj.get(u).contains(v);
    }
}
```

Problem 2: Course Prerequisite

```
class Graph2{
    int v;
    ArrayList<ArrayList<Integer>> adj;

    Graph2(int v){
        this.v=v;
        adj=new ArrayList<>();
        for(int i=0;i<v;i++)
            adj.add(new ArrayList<>());
    }

    void addEdge(int u,int v){
        adj.get(u).add(v);
    }
}
```

Problem 3: DFS BFS

```
void dfs(int node,boolean vis[],ArrayList<ArrayList<Integer>> adj){
    vis[node]=true;
    System.out.print(node+" ");
    for(int x:adj.get(node)){
        if(!vis[x]) dfs(x,vis,adj);
    }
}

void bfs(int s,ArrayList<ArrayList<Integer>> adj){
    boolean vis[]=new boolean[adj.size()];
```

```

Queue<Integer> q=new LinkedList<>();
q.add(s);
vis[s]=true;

while(!q.isEmpty()){
    int node=q.poll();
    System.out.print(node+" ");
    for(int x:adj.get(node)){
        if(!vis[x]){
            vis[x]=true;
            q.add(x);
        }
    }
}
}
}

```

Problem 4: Islands

```

void dfs2(int i,int j,int grid[],boolean vis[]){
    int n=grid.length,m=grid[0].length;
    if(i<0||j<0||i>=n||j>=m||grid[i][j]==0||vis[i][j]) return;

    vis[i][j]=true;
    dfs2(i+1,j,grid,vis);
    dfs2(i-1,j,grid,vis);
    dfs2(i,j+1,grid,vis);
    dfs2(i,j-1,grid,vis);
}

```

```

int countIslands(int grid[]){
    int n=grid.length,m=grid[0].length;
    boolean vis[]=new boolean[n][m];
    int count=0;

    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            if(grid[i][j]==1 && !vis[i][j]){
                dfs2(i,j,grid,vis);
                count++;
            }
        }
    }
    return count;
}

```

Problem 5: Shortest Path

```

void shortestPath(int s,int dest,ArrayList<ArrayList<Integer>> adj){
    int n=adj.size();
    boolean vis[]=new boolean[n];
    int parent[]=new int[n];

    Queue<Integer> q=new LinkedList<>();
    q.add(s);
    vis[s]=true;
    parent[s]=-1;

    while(!q.isEmpty()){
        int node=q.poll();

```

```
for(int x:adj.get(node)){  
    if(!vis[x]){  
        vis[x]=true;  
        parent[x]=node;  
        q.add(x);  
    }  
}  
}
```

```
int cur=dest;  
while(cur!=-1){  
    System.out.print(cur+" ");  
    cur=parent[cur];  
}  
}
```